

Mastering MongoDB

From Shell Basics to Aggregation & Performance — a comprehensive, hands-on developer guide covering all essential MongoDB concepts, patterns, and optimisation techniques in 9 practical modules.

9 MODULES

HANDS-ON EXERCISES

REAL-WORLD EXAMPLES



Show All Commands Ctrl+Shift+P

Go to File Ctrl+P

Find in Files Ctrl+Shift+F

Start Debugging F5

Toggle Terminal Ctrl+`

What You'll Master

This deck is structured as a progressive learning journey — from first shell connection through to production-level performance tuning. Each module builds on the last.



Shell & Data Types

Modules 1–2: mongosh basics and BSON types



CRUD Operations

Modules 3–6: Create, read, update, and delete



Aggregation

Module 7: Multi-stage data pipelines

Performance & Tooling

Modules 8–9: Indexes, explain plans, and utilities

Introduction to the MongoDB Shell (mongosh)

mongosh is the official interactive JavaScript shell for MongoDB. It replaces the legacy **mongo** shell and provides a modern REPL environment for querying, administration, and scripting directly against any MongoDB instance — local or remote.

Connecting to MongoDB

```
// Local instance (default port)
mongosh
```

```
// Local with explicit URI
mongosh
"mongodb://localhost:27017"
```

```
// Remote Atlas cluster
mongosh
"mongodb+srv://user:pass@cluster.mongodb.net/mydb"
```

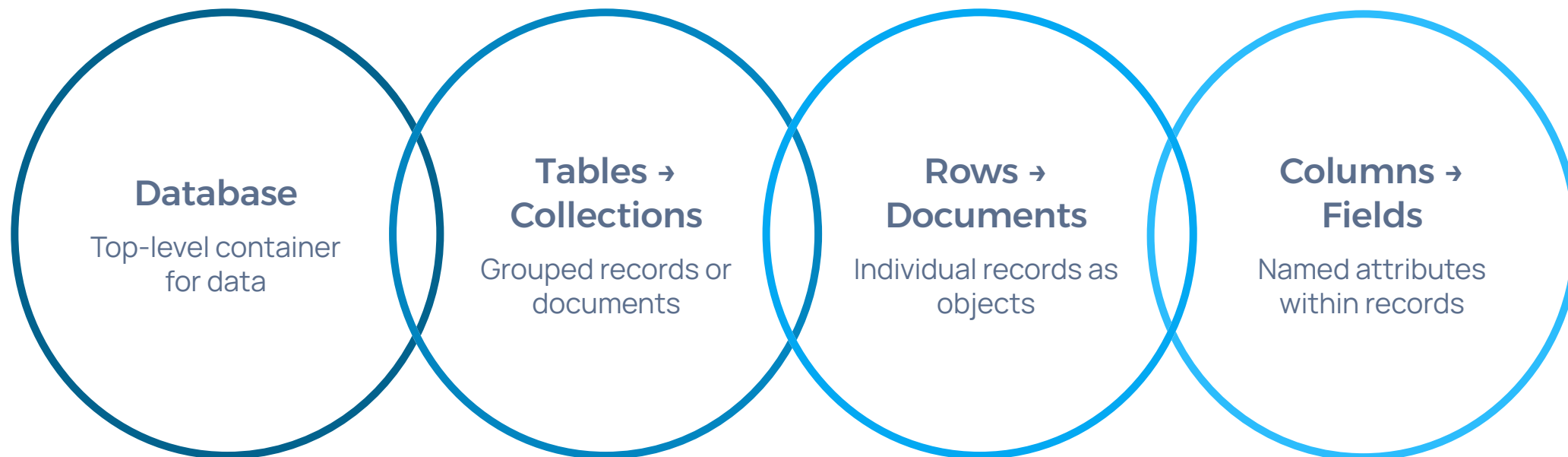
Essential Navigation Commands

```
show dbs // List all databases
use tech_store // Switch/create DB
show collections // List collections
db.stats() // DB metadata & size
db.getName() // Current DB name
```



SQL vs. MongoDB – Conceptual Mapping

MongoDB uses a document model rather than relational tables. Understanding this mapping is essential before writing your first query.



A MongoDB **document** is a JSON-like object (BSON under the hood) that can contain nested objects and arrays — far more flexible than a fixed-schema row. Collections are schema-optional, meaning each document can have a different shape.

❏ Think of a Collection as a table that never complains about missing columns.

Shell Basics Practice

Your Tasks

1 Connect to a local MongoDB instance

Use the default URI and confirm the shell prompt appears.

2 Create the `tech_store` database

Switch into the database using the correct shell command.

3 Verify the active database

Print the current database name to confirm the switch.

Solution Key

```
// 1. Connect to local instance
mongosh "mongodb://localhost:27017"

// 2. Switch to (or create) tech_store
use tech_store

// 3. Verify the active database
db.getName()
// → "tech_store"

// Bonus: confirm it exists
show dbs
```

 Note: MongoDB only physically creates the database once you insert the first document.

Primary MongoDB Data Types (BSON)

MongoDB stores data as **BSON** (Binary JSON) — a binary-encoded superset of JSON that adds richer type information. Choosing the correct BSON type matters for storage efficiency, query accuracy, and sort order.

Strings & Numbers

String — UTF-8 text

NumberInt — 32-bit integer

NumberLong — 64-bit integer

Double — 64-bit float

Decimal128 — high-precision decimal

Identifiers & Time

ObjectId — 12-byte unique `_id`

Date — milliseconds since epoch

Timestamp — internal replication use

Boolean — true / false

Null — absence of value

Complex Types

Array — ordered list of values

Embedded Object — nested document

BinData — binary blobs

Regex — regular expressions

```
// Complete BSON document showcasing all major types
```

```
{  
  _id: ObjectId("64a1f3c9e4b0a12d3c4e5f6a"),  
  name: "Wireless Headphones", // String  
  price: NumberDecimal("49.99"), // Decimal128  
  stock: NumberInt(120), // NumberInt  
  sold: NumberLong(4500), // NumberLong  
  rating: 4.7, // Double  
  inStock: true, // Boolean  
  addedAt: new Date("2024-01-15"), // Date  
  tags: ["audio", "wireless", "sale"], // Array  
  specs: { color: "black", weight: 250 }, // Embedded Object  
  discount: null // Null  
}
```

Basic Create & Read Operations

The foundation of any MongoDB application — inserting documents into a collection and querying them back out. MongoDB's `find()` and projection syntax give you fine-grained control over what data is returned.

Inserting Documents

```
// Insert one document
db.products.insertOne({
  name: "Laptop",
  category: "electronics",
  price: 999.99,
  inStock: true
})

// Insert multiple documents
db.products.insertMany([
  { name: "Mouse", category: "electronics",
    price: 29.99, inStock: true },
  { name: "Desk", category: "furniture",
    price: 349.00, inStock: false }
])
```

Reading & Projecting

```
// Find all electronics
db.products.find(
  { category: "electronics" },
  { name: 1, price: 1, _id: 0 }
)

// Find one specific document
db.products.findOne(
  { name: "Laptop" }
)

// Pretty-print results
db.products.find().pretty()
```

 **Projection tip:** Use `1` to include a field and `0` to exclude it. You cannot mix inclusion and exclusion (except for `_id`).

Advanced Queries & Comparison Operators

MongoDB's query language is rich and expressive. Comparison, logical, element, and array operators can be combined to build precise, efficient queries without writing a single JOIN.

Comparison & Logical

```
// Price between 20 and 100
db.products.find({
  price: { $gte: 20, $lte: 100 }
})

// Price range OR high rating
db.products.find({ $or: [
  { price: { $gte: 20, $lte: 100 } },
  { rating: { $gte: 4.5 } }
]})

// Category in a list
db.products.find({
  category: { $in: ["electronics","audio"] }
})
```

Element & Array Operators

```
// Field must exist and be a Number
db.products.find({
  discount: { $exists: true,
    $type: "number" }
})

// Array contains BOTH values
db.orders.find({
  items: { $all: ["apple","banana"] }
})

// Array element matches conditions
db.orders.find({
  items: { $elemMatch:
    { qty: { $gt: 5 }, price: { $lt: 2 } }
  }
})
```


Updating Documents

MongoDB provides atomic update operators that modify only specific fields without replacing the entire document. Understanding the difference between `updateOne`, `updateMany`, and `replaceOne` is critical for safe, predictable writes.

Field Update Operators

```
// $set and $inc together
db.products.updateOne(
  { name: "Laptop" },
  {
    $set: { lastUpdated: new Date() },
    $inc: { stock: 10 }
  }
)

// $mul: multiply a field value
db.products.updateMany(
  { category: "furniture" },
  { $mul: { price: 1.15 } }
)

// $unset: remove a field entirely
db.products.updateMany(
  {},
  { $unset: { temporary_note: "" } }
)
```

Array Update Operators

```
// $push: append to array
db.products.updateOne(
  { name: "Laptop" },
  { $push: { tags: "refurbished" } }
)

// $addToSet: push WITHOUT duplicates
db.products.updateOne(
  { name: "Laptop" },
  { $addToSet: { tags: "sale" } }
)

// $pull: remove matching element
db.products.updateOne(
  { name: "Laptop" },
  { $pull: { tags: "old-tag" } }
)
```

⚠ **Common Pitfall:** Using `replaceOne()` replaces the *entire* document (except `_id`). Always use update operators like `$set` when you only want partial updates.

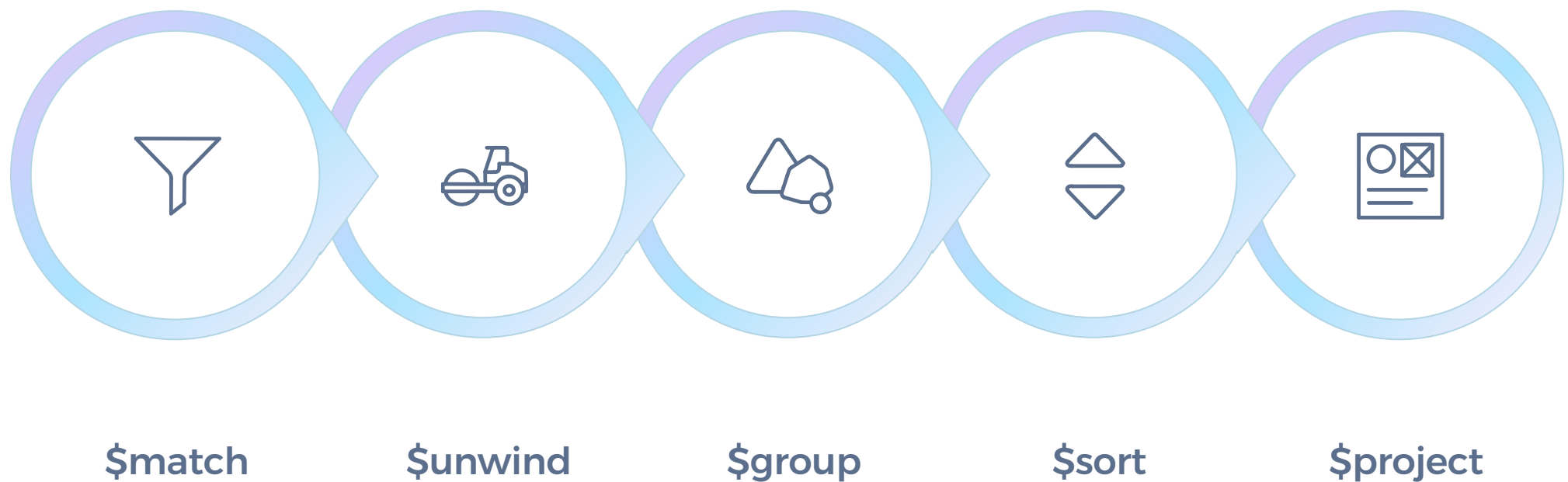
MODULE 6

Delete Operations

Deletes in MongoDB are permanent — there is no built-in undo. Always follow the golden rule: **test your filter with**

The Aggregation Framework

The Aggregation Framework is MongoDB's most powerful data-processing tool. Data flows through a **pipeline** — an ordered array of stages, each transforming the input documents before passing results to the next stage.



Pipeline 1: Revenue per Category

```
db.orders.aggregate([
  { $match: { status: "completed" } },
  { $group: {
    _id: "$category",
    total: { $sum: "$amount" },
    avgOrder: { $avg: "$amount" }
  }},
  { $sort: { total: -1 } },
  { $limit: 5 }
])
```

Pipeline 2: \$lookup (Left Join)

```
db.orders.aggregate([
  { $lookup: {
    from: "users",
    localField: "userId",
    foreignField: "_id",
    as: "userDetails"
  }},
  { $unwind: "$userDetails" },
  { $project: {
    orderId: 1,
    amount: 1,
    "userDetails.name": 1
  } }
])
```

Indexes & Performance Optimisation

Without an index, MongoDB performs a **COLLSCAN** — scanning every document in a collection. Indexes create a B-Tree data structure that allows MongoDB to jump directly to matching documents, dramatically reducing query time on large collections.

Creating & Managing Indexes

```
// Single-field index (ascending)
db.products.createIndex({ price: 1 })

// Compound index: category ASC, price DESC
db.products.createIndex(
  { category: 1, price: -1 }
)

// Text index for full-text search
db.products.createIndex(
  { name: "text", description: "text" }
)

// List all indexes on a collection
db.products.getIndexes()

// Drop a specific index
db.products.dropIndex(
  "category_1_price_-1"
)
```

Diagnose with explain()

```
// Run execution plan analysis
db.products.find(
  { category: "electronics",
    price: { $lt: 500 } }
).explain("executionStats")

// Key fields to look for in output:
// winningPlan.stage:
// "COLLSCAN" → no index used
```

Essential MongoDB Utilities & Tools

Beyond the shell, MongoDB ships with a suite of powerful command-line utilities for backup, restore, import/export, and diagnostics — plus a visual GUI (Compass) and a cloud CLI (Atlas) for modern workflows.



`mongodump` / `mongorestore`

Binary backup and restore tool. Creates BSON dumps of entire databases or individual collections. Best for full backup/DR scenarios.



`mongoexport` / `mongoimport`

JSON and CSV-based import/export. Human-readable output — ideal for data migrations, seeding, and sharing datasets between teams.



`db.currentOp()` / `killOp()`

Real-time diagnostics. Lists currently running operations and allows you to terminate long-running or stuck queries by their `opid`.



Compass & Atlas CLI

Compass provides a visual query builder, schema analyser, and index advisor. Atlas CLI manages cloud clusters, backups, and users from the terminal.

Utilities in Practice – Command Reference

Export & Import

Export a collection **to** JSON

```
mongoexport \  
--uri="mongodb://localhost:27017" \  
--db=tech_store \  
--collection=products \  
--out=products.json
```

Import a JSON file **into** a collection

```
mongoimport \  
--uri="mongodb://localhost:27017" \  
--db=test_db \  
--collection=users \  
--file=users.json \  
--jsonArray
```

Import **from** CSV

```
mongoimport \  
--db=analytics \  
--collection=events \  
--type=csv \  
--headerline \  
--file=events.csv
```

Backup & Diagnostics

Backup entire database

```
mongodump \  
--uri="mongodb://localhost:27017" \  
--db=production_db \  
--out=/backups/$(date +%F)
```

Restore from backup

```
mongorestore \  
--uri="mongodb://localhost:27017" \  
--db=production_db \  
/backups/2024-01-15/production_db
```

List active operations

```
db.currentOp()
```

Kill a slow operation (opid from above)

```
db.killOp(12345)
```

Hands-on Exercises: Modules 2-4

Complete solution keys for the BSON types, CRUD, and advanced query modules.

Module 2 – BSON Types

```
// Correct type choices:
// Price $19.99 → NumberDecimal("19.99")
// Stock 500 → NumberInt(500)
// Purchase date → new Date()
// Tag list → Array ["tag1","tag2"]

// User profile document:
{
  _id: ObjectId(),
  name: "Jane Doe",
  email: "jane@example.com",
  address: {
    street: "123 Main St",
    city: "Austin",
    zip: "78701"
  },
  hobbies: ["cycling","reading","cooking"],
  joinedAt: new Date()
}
```

Module 3 – Basic CRUD

```
// Insert two customers
db.customers.insertMany([
  { name:"Alice", email:"a@x.com",
    city:"NYC" },
  { name:"Bob", email:"b@x.com",
    city:"LA" }
])

// Find all customers named Alice,
// excluding _id from output
db.customers.find(
  { name: "Alice" },
  { _id: 0 }
)

// Module 4 — Advanced Queries
db.products.find({
  price: { $gt: 50 },
  inStock: true
})
db.orders.find({
  items: { $all:["apple","banana"] }
})
```

Hands-on Exercises: Modules 5–6

Solution keys for update and delete operations – the most operationally sensitive commands in MongoDB.

Module 5 – Updates

```
// Increase all furniture prices by 15%
db.products.updateMany(
  { category: "furniture" },
  { $mul: { price: 1.15 } }
)

// Remove "temporary_note" from all docs
db.products.updateMany(
  {},
  { $unset: { temporary_note: "" } }
)

// Add to array without duplicates
db.products.updateOne(
  { name: "Laptop" },
  { $addToSet: { tags: "clearance" } }
)
```

Module 6 – Deletes

```
// Delete logs older than 30 days
const cutoff = new Date(
  Date.now() - 30*24*60*60*1000
)
// Preview first!
db.logs.find({
  createdAt: { $lt: cutoff }
}).count()
// Then delete:
db.logs.deleteMany({
  createdAt: { $lt: cutoff }
})

// Safely drop a test collection
// Step 1: verify name
show collections
// Step 2: drop it
db.temp_data.drop()
```

 **Best Practice:** Always run a `countDocuments()` check before any `deleteMany()` in production. The numbers should match your expectations exactly before proceeding.

Hands-on Exercises: Modules 7-9

Solution keys for aggregation pipelines, index creation, and utility commands.

Module 7 – Aggregation

```
// Pipeline: avg order per customer
db.orders.aggregate([
  { $match: { status:"completed" } },
  { $group: {
    _id: "$customerId",
    avg: { $avg: "$total" },
    count: { $sum: 1 }
  }},
  { $sort: { avg: -1 } }
])
```

```
// Unwind tags and count each
db.products.aggregate([
  { $unwind: "$tags" },
  { $group: {
    _id: "$tags",
    total: { $sum: 1 }
  }},
  { $sort: { total: -1 } }
])
```

Modules 8-9 – Indexes & Tools

```
// Compound index: lastName + firstName
db.customers.createIndex(
  { lastName: 1, firstName: 1 }
)
// Run explain to verify IXSCAN
db.customers.find(
  { lastName: "Smith" }
).explain("executionStats")
```

```
// Backup production_db
mongodump \
--db=production_db \
--out=./backups
```

```
// Import users.json into test_db
mongoimport \
--db=test_db \
--collection=users \
--file=users.json \
--jsonArray
```

Pro Tips, Pitfalls & Best Practices

Pro Tips

- Use `.explain("executionStats")` on any slow query — look for `IXSCAN` in the winning plan.
- Use `$addToSet` instead of `$push` when array uniqueness matters.
- Prefer `insertMany()` over looped `insertOne()` calls — it's a single network round-trip.
- Use the ESR rule (Equality → Sort → Range) when designing compound indexes.

Common Pitfalls

- `replaceOne()` overwrites the entire document — always use `$set` for partial updates.
- Forgetting `multi: true` in older drivers — in mongosh, `updateMany()` handles this automatically.
- Running `deleteMany({})` without a filter removes *all* documents in the collection.
- Indexes speed up reads but slow down writes — avoid over-indexing write-heavy collections.

Best Practices

- Always test destructive queries with `find()` or `countDocuments()` first.
- Use `Decimal128` for monetary values — never `Double`.
- Schedule regular `mongodump` backups and test restores periodically.
- Use Atlas Search or text indexes rather than regex on large collections.

Quick Reference: Operator Cheat Sheet

A consolidated reference for the most commonly used MongoDB operators across all modules.

Query Operators

Operator	Meaning
<code>\$eq</code> / <code>\$ne</code>	Equal / Not equal
<code>\$gt</code> / <code>\$gte</code>	Greater than / or equal
<code>\$lt</code> / <code>\$lte</code>	Less than / or equal
<code>\$in</code> / <code>\$nin</code>	In / Not in array
<code>\$and</code> / <code>\$or</code>	Logical AND / OR
<code>\$exists</code>	Field presence check
<code>\$all</code>	Array contains all values
<code>\$elemMatch</code>	Array element conditions

Update & Aggregation Operators

Operator	Meaning
<code>\$set</code> / <code>\$unset</code>	Set / Remove a field
<code>\$inc</code> / <code>\$mul</code>	Increment / Multiply
<code>\$push</code>	Append to array
<code>\$addToSet</code>	Append, no duplicates
<code>\$pull</code> / <code>\$pop</code>	Remove from array
<code>\$match</code>	Pipeline filter stage
<code>\$group</code>	Accumulate / aggregate
<code>\$lookup</code>	Left outer join

You've Mastered MongoDB

From your first `mongosh` connection to compound indexes and aggregation pipelines — you now have the foundational knowledge to build, query, and optimise MongoDB at a production level.

Next Steps

Explore **MongoDB Atlas** for managed cloud hosting, automatic backups, and global clusters.

Go Deeper

Study **transactions**, **change streams**, and **Atlas Search** for advanced use cases.

Practice Daily

The fastest way to internalise these patterns is to use them on real data — build a project today.



All 9 modules complete. Solution keys for every hands-on exercise are included in slides 15–17. Keep this deck as a living reference.